Registro de notas

Práctica 2 de Arrays · Desarrollo Web en Entorno Cliente · 2º DAW

Memoria de la solución. El proyecto permite gestionar los alumnos de una clase, ordenar y filtrar sus notas y calcular estadísticas mediante arrays. Los archivos del proyecto se entregan junto a esta memoria.

Fase 1 Exploración de arrays

Predicciones orientativas para comparar con la ejecución. Los bloques se ejecutan en el orden indicado; en el bloque D se crea de nuevo n antes de cada operación.

Línea	Predicción	Resultado observado
1 · a[0]	"x"	"x"
2 · a[3]	undefined	undefined
3 · a.length	3	3
4 · [1, "dos", true].length	3	3
5 · const c = [1]; c.push(2); c	[1, 2]	push devuelve 2; c queda [1, 2]
6 · q.push(4); mirar p	p también cambia	push devuelve 4; p queda [1, 2, 3, 4]
7 · let r = [...p]; r.push(5); mirar p	p no cambia	r queda [1, 2, 3, 4, 5]; p sigue [1, 2, 3, 4]
8 · [10, 9, 1].sort()	[1, 10, 9]	[1, 10, 9]
9 · sort((x, y) => x - y)	[1, 9, 10]	[1, 9, 10]
10 · n.push(4)	Devuelve longitud 4 y cambia n	4; n = [3, 1, 2, 4]
11 · n.pop()	Devuelve 2 y cambia n	2; n = [3, 1]
12 · n.slice(0, 2)	Copia [3, 1]; n no cambia	[3, 1]; n = [3, 1, 2]
13 · n.map(x => x * 2)	Copia [6, 2, 4]; n no cambia	[6, 2, 4]; n = [3, 1, 2]

1. a[3] devuelve undefined: tres elementos ocupan los índices 0, 1 y 2; el índice 3 no existe.

2. q = p copia la referencia, por lo que ambas variables apuntan al mismo array. [...p] crea otro array y añadir un elemento a r no cambia p. Es una copia superficial: si los elementos fueran objetos, seguirían siendo referencias a los mismos objetos.

3. sort sin comparador compara representaciones de texto; "10" va antes que "9". Para números se usa (x, y) => x - y.

4. push y pop modifican el original; slice y map devuelven arrays nuevos. const impide reasignar la variable, pero no impide cambiar el contenido del array.

Fase 2 Añadir, quitar, buscar y ordenar

Función	Efecto en la lista
anadirAlumno	Cambia: push añade al final
quitarUltimo	Cambia: pop retira el último
buscarPorNombre	No cambia: find consulta
insertarEn	Cambia: splice inserta sin borrar
borrarPorNombre	Cambia si encuentra un alumno: splice lo borra
ordenarPorNota	No cambia: sort se aplica a [...lista]
ordenarPorNombre	No cambia: sort se aplica a [...lista]

1. Se distingue por el método empleado y se comprueba comparando la lista antes y después; las pruebas verifican tanto contenido como identidad de las copias.

2. sort modifica el array sobre el que se ejecuta. La copia impide reordenar la lista de origen.

3. Sin comparador, las notas 10, 9 y 1 quedarían [1, 10, 9], al ordenarse como texto.

Corrección del enunciado: con los datos iniciales, el orden correcto empieza por Lucía (10), Marta (9,25) y Hugo (9). La indicación de que Hugo debe quedar segundo contradice la nota de Marta.

Fase 3 Estadísticas de la clase

Para Ana 7,5, Luis 4, Marta 9,25 y Pablo 5: media 6,4375; máxima 9,25; aprobados Ana, Marta y Pablo; hayAlgunDiez false; todosAprobados false; ranking de dos: Marta y Ana.

Función con lista vacía	Resultado
notaMedia([])	0
notaMaxima([])	null
aprobados([])	[]
nombresAprobados([])	[]
hayAlgunDiez([])	false
todosAprobados([])	true
ranking([], 2)	[]

1. reduce necesita un acumulador inicial si no hay elementos. [].reduce((a, b) => a + b) lanza TypeError: Reduce of empty array with no initial value. Con el valor inicial 0, devuelve 0. Si notaMedia conserva la comprobación previa de longitud, la lista vacía devuelve 0 antes de llegar a reduce; para observar el error se ejecuta reduce directamente, sin esa protección.

2. every en una lista vacía devuelve true porque no existe ningún elemento que incumpla la condición. Esto no significa que haya alumnos aprobados; significa que no hay ningún suspenso.

3. map transforma cada elemento y mantiene la cantidad: aprobados(lista).map(a => a.nombre) devuelve nombres. filter selecciona elementos: lista.filter(a => a.nota >= 5) devuelve solo los aprobados.

Ninguna función de estadísticas altera el array original. Las copias son superficiales: no se editan los objetos de alumnos.

Fase 4 Tabla dinámica

1. filtrarAlumnos y ordenarAlumnos devuelven copias para que cambiar la vista no elimine ni reordene permanentemente los datos de trabajo. El filtro «todos» también devuelve un array diferente.

2. Si ordenarAlumnos modificara el original, cambiaría el orden de inserción, y futuras operaciones como quitar el último alumno actuarían sobre el último de la vista ordenada en lugar del último añadido.

pintarTabla es la única función que crea filas. Los selectores llaman a esa función; añadir y borrar llaman además a pintarResumen. El resumen usa la lista completa. Se rechazan nombres vacíos, duplicados y notas vacías, no numéricas o fuera de 0 a 10. Los nombres duplicados se comparan sin mayúsculas y sin espacios exteriores, pero conservando los acentos. Los mensajes se escriben sin alert. Se usa textContent para que un nombre se muestre como texto y no como código HTML.

La aplicación guarda los cambios en memoria durante la sesión; recargar restaura los datos iniciales. No se ha añadido almacenamiento persistente porque no forma parte del enunciado.

Fase 5 Depuración y documentación

console.table(alumnos) presenta una fila por alumno y columnas de índice, nombre y nota. console.table(alumnos.map(a => a.nota)) presenta los índices y únicamente los valores numéricos de las notas. Los ejemplos se ejecutan sobre copias para no alterar el registro.

Fallo	Qué pasaba y por qué	Corrección	Herramienta
1 · sort sin comparador	Devuelve [1, 10, 5, 9], porque compara texto	notas.sort((a, b) => a - b) produce [1, 5, 9, 10]	console.log
2 · copia = alumnos	Ordenar copia también reordena alumnos: ambas variables referencian el mismo array	const copia = [...alumnos]; después ordenar copia	console.table antes y después; comparación copia === alumnos
3 · reduce sin valor inicial	La lista vacía lanza TypeError: Reduce of empty array with no initial value	notas.reduce((a, b) => a + b, 0) devuelve 0	Mensaje de la consola
4 · i <= alumnos.length	Intenta leer alumnos[alumnos.length].nombre, pero ese elemento es undefined	Usar i < alumnos.length o for (const alumno of alumnos)	console.log de los nombres y mensaje TypeError

Cada función de notas.js incluye descripción, @param, @returns y explicación de si modifica la lista. Los cuatro fragmentos con errores pueden repetirse con node depuracion.js. Los errores están capturados y no detienen la aplicación.

Ejemplo de documentación JSDoc

```
/** Calcula la media sin redondear. No modifica la lista.
 * @param {{nombre:string, nota:number}[]} lista Alumnos.
 * @returns {number} Media o 0 si la lista está vacía.
 */
function notaMedia(lista) {
  return lista.length
    ? lista.reduce((suma, a) => suma + a.nota, 0) / lista.length
    : 0;
}
```

Verificación y entrega

Resultados ejecutados: 71 de 71 pruebas de funciones y 17 de 17 comprobaciones de interfaz con DOM simulado. La suma de las diez notas es 61,5; la media es 6,15; hay siete aprobados y la máxima es 10. Una expectativa inicial de media 6,45 fue detectada por las pruebas y corregida tras sumar los datos; la función devolvía correctamente 6,15.

El proyecto incluye cinco commits locales, uno por fase, y archivos de evidencias de consola. evidencias.html calcula ejemplos y pruebas cuando se abre en un navegador. Los resultados de Node.js y del DOM simulado no sustituyen una inspección visual del navegador.

No se incluyen capturas reales de navegador ni se afirma que la web esté publicada. La publicación en GitHub Pages queda pendiente de un repositorio propio de destino: la conexión disponible solo expone un repositorio ajeno a esta práctica y no permite crear un repositorio nuevo mediante las herramientas actuales. La aplicación sí puede abrirse directamente desde index.html.

Abrir y comprobar el proyecto

1. Descomprime el ZIP y abre la carpeta registro-notas en Visual Studio Code.

2. Abre index.html en el navegador. Los archivos CSS y JavaScript se cargan desde esa misma carpeta.

3. Abre evidencias.html para ver ejemplos y pruebas calculados en el navegador.

4. En la terminal, ejecuta git log --oneline para ver los cinco commits. Con Node.js instalado, ejecuta node pruebas.js y node verificar-interfaz.cjs.

Archivos principales

Archivo	Contenido
index.html	Formularios, tabla y resumen
estilos.css	Diseño adaptable y estados visuales
notas.js	Datos iniciales y funciones con JSDoc
app.js	Eventos, validación y dibujo de la tabla
notas.md	Respuestas y resultados de las cinco fases
README.md	Instrucciones y clasificación de funciones
pruebas.js	71 pruebas de funciones
verificar-interfaz.cjs	17 comprobaciones con DOM simulado